编译原理实验报告

实验 7：用 Lex 和 Yacc 设计语法分析器 1

实验 8：用 Lex 和 Yacc 设计语法分析器 2

组员姓名：姚嫣如（21080012078）

仇冠文（23020013005）

高笑颜（23020007026）

陈怡冰（23020007007)

陆晨辰(22110031014)

完成日期：　2026　年　7　月　1　日

实验目录

实验 7：用 Lex 和 Yacc 设计语法分析器 1 1

一、 实验目的 1

二、 实验内容 1

三、 实验要求 1

四、 实验过程及重点内容 1

五、 实验结果 10

六、 实验中遇到的问题、难点及解决方案 10

七、 感想和收获 13

实验 8：用 Lex 和 Yacc 设计语法分析器 2 13

一、 实验目的 13

二、 实验内容 13

三、 实验要求 14

四、 实验过程及重点内容 14

五、 实验结果 29

六、 实验中遇到的问题、难点及解决方案 29

七、 感想和收获 32

实验 7：用 Lex 和 Yacc 设计语法分析器 1

一、 实验目的

学习如何使用 Yacc 设计一个语法分析器，并与用 lex 写的词法分析器链接起来。

二、 实验内容

使用 yacc 为实验 3 所给的语言写一个语法分析器（你可以重新设计该语言的文法，但不能改变语言）。

其中，词法分析使用实验 4 中已完成的词法分析器（即，你需要将本实验的语法分析器和实验 4 的词法分析器链接起来）。

三、 实验要求

1、在本实验已有代码文件（见代码文件说明）的基础上，完善语法分析器 parser.y 并进行编译，能够正确响应输入内容，并输出预期结果：输入为一段源程序文本；输出语法分析是否成功的提示（“Parse Successful!”或“Parse Failed!”）。

2、在语法分析中不能出现任何的冲突（移进-归约或归约-归约冲突），或者虽然有冲突，但是你能够说清楚冲突是如何解决的。

3、可以在本实验已有代码文件（见代码文件说明）的基础上修改，也可以自行修改这些代码文件的内容。如果你尚不能将实验 3 的词法分析器和本实验的语法分析器链接起来，可以暂时使用已有文件 lex.yy.c 中给出的词法分析器（前提是你的语法分析器中终结符名的定义与该例子中的终结符定义相同）。

4、你的程序应能通过本节中的所有测试用例。

四、 实验过程及重点内容

4.1 实验过程

1、当前有什么

- **基础框架：** 基于 Appel 的 Tiger 编译器前端，提供了 parsetest.c 作为主控程序，errormsg.c/h 负责错误定位。
- **初步文法：** 提供了一个极简的文法模板（仅支持单变量声明和单赋值语句），无法处理复杂的嵌套结构。

- **预设词法：** 提供了编译好的 lex.yy.c，但为了实现自定义逻辑和位置跟踪，需要自行编写 parser.l。

2、实验步骤

（1）补全词法分析器 parser.l

- **正则表达式定义：**使用正则表达式定义了程序中所有的记号（Token），包括关键字（PROGRAM, IF, WHILE 等）、运算符（:=, <=, +, * 等）及界符。
- **位置同步算法：**在每个动作中调用 adjust() 函数，将词法单元的绝对位置传递给全局变量 EM_tokPos，并处理 \n 以调用 EM_newline() 更新行号。
- **属性值传递机制：**通过 yylval 联合体将标识符的名字（字符串）和数值（整型/浮点型）传递给语法分析器。

（2）重构语法分析器 parser.y：

- **定义 Token 与属性类型：**在 %union 中声明了 ival, fval, sval 类型，并为 ID, INT, FLOAT 指定了对应的属性。
- **设计递归文法：**为了通过复杂的测试用例（如测试 3），将简单的文法改写为支持列表和嵌套的上下文无关文法（CFG）。包括：
  - idlist：支持逗号分隔的**多变量声明**。
  - stmts：支持分号分隔的语句序列。
  - exp：支持加减乘除算术运算和比较运算。

```
整体结构：
program -->    PROGRAM ID ; vardec BEGINN stmts END .

变量声明：
vardec -->     VAR declist | ε
declist -->    dec | declist dec
dec -->       idlist : type ;
idlist -->     ID | idlist , ID
type -->       INTEGER | REAL

语句：
stmts -->          stmt | stmts ; stmt | ε
stmt -->           assignment_stmt | if_stmt | while_stmt |
                   compound_stmt | ε
assignment_stmt --> ID := exp
if_stmt -->       IF exp THEN stmt | IF exp THEN stmt ELSE stmt
while_stmt -->    WHILE exp DO stmt
compound_stmt --> BEGINN stmts END
```

```
表达式:
exp -->       ID | INT | FLOAT | ( exp ) | - exp
exp -->       exp + exp | exp - exp | exp * exp | exp / exp
exp -->       exp < exp | exp <= exp | exp > exp | exp >= exp |
              exp = exp | exp <> exp
```

（3）解决文法冲突

- 通过 *%left* 和 *%right* 声明运算符的优先级与结合性，解决了算术表达式的二义性冲突。
- 使用 *%nonassoc* 解决了经典的“悬空 Else”移进-归约冲突。

4.2 重点内容

1、词法与语法的链接机制（The Bridge）

实验的核心在于理解 Lex 与 Yacc 如何协同工作。

Bison 根据 parser.y 生成 parser.tab.h，该文件定义了所有 Token 的整数编号。词法分析器 parser.l 必须包含此头文件，以便识别 return ID; 等语句。同时，利用 yylval 这一共享联合体，词法分析器不仅能告诉语法分析器“看到了什么”，还能传递“具体值是什么”。

2、递归文法处理列表结构

为了解析 i, j, k: INTEGER; 这种结构，文法必须设计成递归形式：

```
idlist : ID                     /* 基准情况 */
       | idlist COMMA ID        /* 递归情况 */
       ;
```

这种设计模式同样应用于语句序列（stmts）和变量声明列表（declist），使得分析器能够处理任意长度的输入，突破了初始框架中“只能处理单个记号”的局限。

3、二义性消除与冲突处理

- **算术运算：**表达式规则 exp : exp PLUS exp 会导致 25 个以上的移进-归约冲突。通过在声明区添加 %left PLUS MINUS 和 %left TIMES DIVIDE（位置越靠后优先级越高），Bison 能够自动选择正确的移进动作，保证先乘除后加减。
- **悬空 Else：**当 IF 语句嵌套时，ELSE 默认与最近的 IF 匹配。通过声明 %nonassoc LOWER_THAN_ELSE 和 %nonassoc ELSE，并结合 %prec 标

记，显式告诉分析器在遇到 ELSE 时优先进行移进（Shift）而非归约（Reduce）。

4、精确的错误定位算法

实验要求输出 temp.in:1.55 这种形式的报错。其背后的逻辑是：

- parser.l 通过 adjust() 记录每个 Token 在文件中的绝对字符偏移量。
- errormsg.c 维护一个记录每行起始位置的链表。
- 当 yyerror 被触发时，系统用“当前绝对位置 - 行起始位置”计算出列号。
- 这要求词法分析器在匹配任何字符（包括空白符）时都不能漏掉位置更新。

5、文法的健壮性：空产生式（Empty Productions）

在处理 BEGIN ... END 之间的语句块时，为了处理可能出现的空语句或末尾多余的分号，文法引入了/* empty */ 产生式。这使得分析器在遇到 BEGIN END（无语句）或 i:=1; END（带分号的末尾）时不会因无法匹配而报错，增强了分析器的容错能力。

4.3 实验代码

4.3.1 编写 parser.l 词法分析器

```
%{
#include <string.h>
#include <stdlib.h>
#include "util.h"
#include "errormsg.h"
#include "parser.tab.h"
int charPos = 1;
void adjust(void) {
    EM_tokPos = charPos;
    charPos += yyleng;
}
%}
delim       [ \t\r]
ws          {delim}+
letter      [A-Za-z]
digit       [0-9]
id          {letter}({letter}|{digit})*
int_num     {digit}+
float_num   ({digit}+\.{digit}*)|(\.{digit}+)
```

```
/* 注意：这里必须用 %x，而不是 %s */
%x COMMENT
%%
\n              {adjust(); EM_newline();}
{ws}            {adjust();}
 /* 注释 */
"(*"            {adjust(); BEGIN COMMENT;}
<COMMENT>"*)"   {adjust(); BEGIN INITIAL;}
<COMMENT>\n     {adjust(); EM_newline();}
<COMMENT>.      {adjust();}
 /* 关键字必须放在 ID 前面 */
"PROGRAM"       {adjust(); return PROGRAM;}
"VAR"           {adjust(); return VAR;}
"INTEGER"       {adjust(); return INTEGER;}
"REAL"          {adjust(); return REAL;}
"BEGIN"         {adjust(); return BEGINN;}
"END"           {adjust(); return END;}
"IF"            {adjust(); return IF;}
"THEN"          {adjust(); return THEN;}
"ELSE"          {adjust(); return ELSE;}
"WHILE"         {adjust(); return WHILE;}
"DO"            {adjust(); return DO;}
 /* 运算符和界符 */
":="            {adjust(); return ASSIGN;}
"<>"            {adjust(); return NEQ;}
"<="            {adjust(); return LE;}
">="            {adjust(); return GE;}
":"             {adjust(); return COLON;}
";"             {adjust(); return SEMICOLON;}
","             {adjust(); return COMMA;}
"."             {adjust(); return PERIOD;}
"("             {adjust(); return LPAREN;}
")"             {adjust(); return RPAREN;}
"+"             {adjust(); return PLUS;}
"-"             {adjust(); return MINUS;}
"*"             {adjust(); return TIMES;}
"/"             {adjust(); return DIVIDE;}
"="             {adjust(); return EQ;}
"<"             {adjust(); return LT;}
">"             {adjust(); return GT;}
 /* 浮点数要放在整数前面 */
{float_num}     {adjust(); yylval.fval = atof(yytext); return FLOAT;}
{int_num}       {adjust(); yylval.ival = atoi(yytext); return INT;}
{id}            {adjust(); yylval.sval = String(yytext); return ID;}
```

```
.               {adjust(); EM_error(EM_tokPos, "illegal token: %s", yyt
ext);}
%%
void resetLexState() {
    BEGIN INITIAL;
    charPos = 1;
}
int yywrap() {
    charPos = 1;
    return 1;
}
```

4.3.2 编写 parser.y 语法分析器

第一种写法：

```
%{
#include <stdio.h>
#include <stdlib.h>
#include "util.h"
#include "errormsg.h"
int yylex(void);
int parseSuccess = 0;
void yyerror(char *s)
{
    parseSuccess = 0;
}
%}
%union {
    int ival;
    double fval;
    string sval;
}
%token <sval> ID
%token <ival> INT
%token <fval> FLOAT
%token INTEGER REAL
%token
    COMMA COLON SEMICOLON LPAREN RPAREN PERIOD
    PROGRAM BEGINN END VAR IF WHILE DO
    THEN ELSE
    ASSIGN EQ NEQ LT LE GT GE
    PLUS MINUS TIMES DIVIDE
%start program
%%
program
```

```
    : PROGRAM ID SEMICOLON vardec BEGINN stmtseq END PERIOD
      { parseSuccess = 1; }
    ;
vardec
    : VAR decls
    ;
decls
    : decls decl
    | decl
    ;
decl
    : idlist COLON type SEMICOLON
    ;
idlist
    : ID
    | idlist COMMA ID
    ;
type
    : INTEGER
    | REAL
    ;
stmtseq
    : stmt
    | stmtseq SEMICOLON stmt
    ;
stmt
    : ID ASSIGN exp
    | BEGINN stmtseq END
    | IF cond THEN stmt ELSE stmt
    | WHILE cond DO stmt
    ;
cond
    : exp LT exp
    | exp LE exp
    | exp GT exp
    | exp GE exp
    | exp EQ exp
    | exp NEQ exp
    ;
exp
    : INT
    | FLOAT
    | ID
    | LPAREN exp RPAREN
```

```
    | exp PLUS exp
    | exp MINUS exp
    | exp TIMES exp
    | exp DIVIDE exp
    ;
```

第二种写法：

```
%{
#include <stdio.h>
#include "util.h"
#include "errormsg.h"
int yylex(void);
void yyerror(char *s)
{
    EM_error(EM_tokPos, "%s", s);
}
%}
%union {
    int ival;
    double fval;
    string sval;
}
%token <sval> ID
%token <ival> INT
%token <fval> FLOAT
%token INTEGER REAL
%token COMMA COLON SEMICOLON LPAREN RPAREN PERIOD
%token PROGRAM BEGINN END VAR IF WHILE DO THEN ELSE
%token ASSIGN EQ NEQ LT LE GT GE
%token PLUS MINUS TIMES DIVIDE
%start program
%nonassoc LOWER_THAN_ELSE
%nonassoc ELSE
%left EQ NEQ LT LE GT GE
%left PLUS MINUS
%left TIMES DIVIDE
%right UMINUS
%debug
%%
program
    : PROGRAM ID SEMICOLON vardec BEGINN stmts END PERIOD
    ;
vardec
    : VAR declist
```

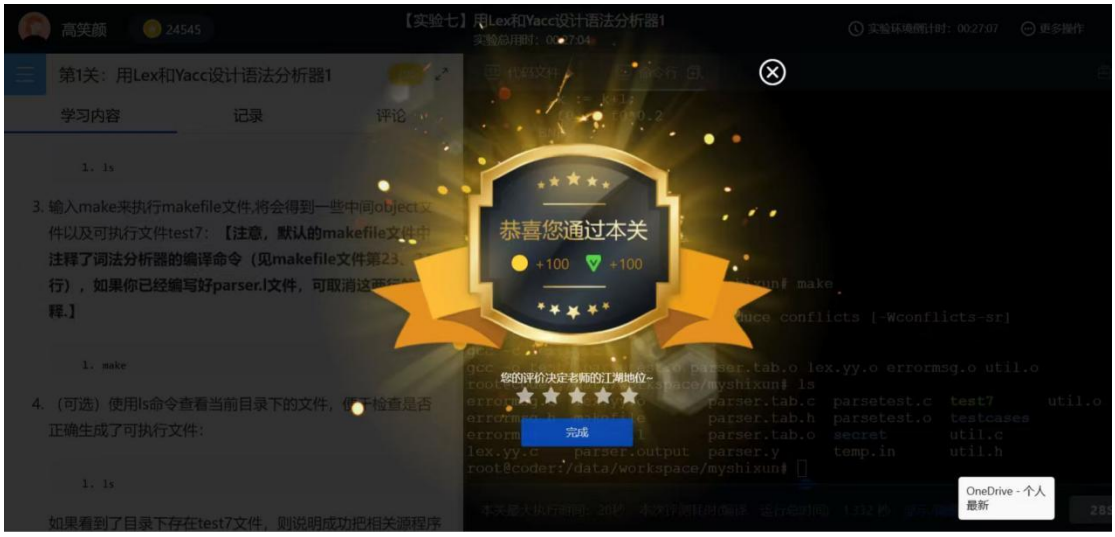
```
    ;
declist
    : dec
    | declist dec
    ;
dec
    : idlist COLON type SEMICOLON
    ;
idlist
    : ID
    | idlist COMMA ID
    ;
type
    : INTEGER
    | REAL
    ;
stmts
    : /* empty */
    | stmtlist
    ;
stmtlist
    : stmt
    | stmtlist SEMICOLON stmt
    | stmtlist SEMICOLON
    ;
stmt
    : assignment_stmt
    | if_stmt
    | while_stmt
    | compound_stmt
    ;
assignment_stmt
    : ID ASSIGN exp
    ;
if_stmt
    : IF exp THEN stmt %prec LOWER_THAN_ELSE
    | IF exp THEN stmt ELSE stmt
    ;
while_stmt
    : WHILE exp DO stmt
    ;
compound_stmt
    : BEGINN stmts END
    ;
```

```
exp
    : exp PLUS exp
    | exp MINUS exp
    | exp TIMES exp
    | exp DIVIDE exp
    | exp LT exp
    | exp LE exp
    | exp GT exp
    | exp GE exp
    | exp EQ exp
    | exp NEQ exp
    | LPAREN exp RPAREN
    | MINUS exp %prec UMINUS
    | INT
    | FLOAT
    | ID
    ;
%%
```

第一种写法中定义了 parseSuccess 让程序其他部分（主要是 main 函数）方便判断整个解析是否成功，而第二种方法直接利用 yyparse()的返回值，两种写法都可以正常完成语法分析。

五、 实验结果

全部组员通过实验。



六、 实验中遇到的问题、难点及解决方案

1、token 名称与 lex.yy.c 不一致

现象：编译时报 'SEMICOLON' undeclared、'PERIOD' undeclared、'BEGINN' undeclared 等错误。

原因：最初照常识写了 SEMI、DOT、BEGIN_，但实际 lex.yy.c 中使用的是不同的名字。

解决方法：通过以下命令从 lex.yy.c 中提取实际 token 名：

```
grep "return " lex.yy.c | grep -v "//"
```

确认实际 token 名为：

符号	token 名
;	SEMICOLON
.	PERIOD
BEGIN 关键字	BEGINN（因 BEGIN 是 flex 内置宏）
REAL 类型	REAL
浮点数	FLOAT

2、链接时 undefined reference to 'yydebug'

现象：errormsg.c 中声明了 extern int yydebug，链接时找不到定义。

原因：yydebug 是 bison 的调试变量，只有在 bison 开启调试模式时才会生成。

解决方法：在 parser.y 的声明区加上 %debug，让 bison 生成 yydebug 变量。

或 在 parser.y 开头%{ ... %}中加入 int yydebug = 0;

3、Parse Successful! 输出两次

现象：make test00 运行后输出了两行 Parse Successful!。

原因：parsetest.c 的 parse() 函数里已经负责打印，parser.y 的 action 里又写了一次 printf。

解决方法：删除 parser.y 中 program 规则里的 printf action。

4、所有测试用例都不通过

现象：补充文法以后，每一个测试都不通过。第一个测试用例错误定位到第 4 行第 1 列，是在给出参数 VAR 的结尾出错。

原因：列出形参的文法有问题，文法会导致列出不同类型的形参时中间的会多一个;的检测。

```
VAR i:INT;;
    j:REAL;
```

解决方法：在 decls 的文法设计中去掉;，让 decl 自己管理;

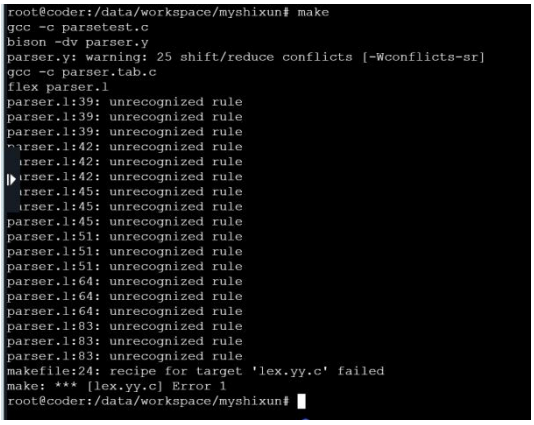
```
vardec   : VAR decls
         ;
decls    : decl | decls decl
         ;
decl     : id_list COLON type SEMICOLON
         ;
```

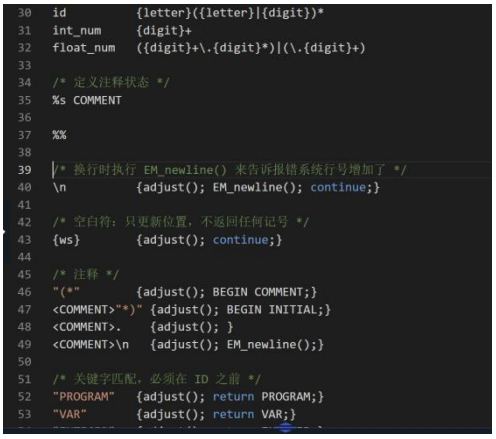
5、注释行报错

现象：在编译时出现报错，报错位置都在注释行

原因：在 Flex（parser.l）文件的规则段（第二个 %% 之后），注释或代码不能顶格写。

解决方法：在注释内容前添加空格。





6、在无字符位置出现报错

现象：在自行测试用例 3 时，反复修改代码，反复出现 temp.in:1.55 报错



原因：第一行的注释中存在英文 Parse Successful，词法分析器把注释里的 Parse 或 Successful 当成了 ID 返回给语法分析器，所以语法分析器一开始没有读到 PROGRAM，而是读到了一个 ID，于是直接 syntax error。

解决方法：将%s COMMENT 修改为 %x COMMENT

- %s COMMENT 定义的是**包含型状态**。在 COMMENT 状态下，没有写状态限定的普通规则仍然会生效，所以注释中的英文单词仍然可能被识别成 ID。
- %x COMMENT 是**排他型状态**，进入注释状态后，只会匹配 <COMMENT> 开头的

规则，普通的 ID、关键字、数字规则不会再生效。

7、不支持多变量同行声明

现象：VAR i, j, k: INTEGER; 解析失败。

原因：初始文法的 decitem 规则只支持单个变量 ID : type ;，没有逗号分隔的 id 列表

解决方法：新增 idlist 规则支持逗号分隔：

```
yacc
decitem : idlist COLON type SEMICOLON ;
idlist  : idlist COMMA ID | ID ;
```

七、 感想和收获

通过本次实验，深刻体会到了词法分析与语法分析之间“环环相扣”的协作逻辑。在实现 Lex 与 Yacc 链接的过程中，理解了如何通过 parser.tab.h 共享记号编号，并利用 yylval 联合体在两者间传递属性值，这让原本抽象的编译器前端理论变得具体可感。

实验中最具挑战性的是对文法二义性的处理。面对“悬空 Else”和算术运算带来的移进-归约冲突，学会了利用优先级声明（%left）和 %prec 标记来引导分析器做出正确决策，极大锻炼了逻辑思维。此外，通过 adjust()函数实现精确到行列号的错误定位，认识到了编译器健壮性的重要。通过这次实验不仅掌握了 Yacc 的编程范式，更让学会了如何设计一套支持递归嵌套的工业级文法，为后续构建抽象语法树（AST）奠定了坚实基础。

实验 8：用 Lex 和 Yacc 设计语法分析器 2

一、 实验目的

学习如何使用 Yacc 设计一个语法分析器，并与用 lex 写的词法分析器链接起来。

二、 实验内容

修改实验 7，给产生式加上动作，动作为生成一棵语法分析树。这棵分析树的结构可以使用或参照本实验已有代码文件（见代码文件说明）中 ast.h 文件中定义的分析树结构。

三、 实验要求

1、在本实验已有代码文件（见代码文件说明）的基础上，完善词法分析器 parser.l 和语法分析器 parser.y 并进行编译，能够正确响应输入内容，并输出预期结果：输入为一段源程序文本；输出为一棵语法分析树。你可以在自己的语法分析器中直接使用 prast.c 已经提供的输出方式（打印分析树的数据结构），但前提是你必须使用 ast.h 提供的分析树结构。

2、在语法分析中不能出现任何的冲突（移进-归约或归约-归约冲突），或者虽然有冲突，但是你能够说清楚冲突是如何解决的。

3、可以在本实验已有代码文件（见代码文件说明）的基础上修改，也可以自行修改这些代码文件的内容。如果你尚不能将实验 4 的词法分析器和本实验的语法分析器链接起来，可以暂时使用已有文件 lex.yy.c 中给出的词法分析器（前提是你的语法分析器中终结符名的定义与该例子中的终结符定义相同）。

4、你的程序应能通过本节中的所有测试用例。

四、 实验过程及重点内容

本次实验是在实验七语法分析器的基础上进一步扩展完成的。实验七的主要任务是判断源程序是否符合文法，而实验八的重点是在语法分析过程中为各个产生式添加语义动作，并在归约时构造对应的抽象语法树，最终输出语法分析树的结构。

实验采用 lex/flex 完成词法分析，使用 yacc/bison 完成语法分析。词法分析器负责将源程序中的关键字、标识符、整数、实数、运算符和界符识别为对应的 token；语法分析器根据文法规则对 token 序列进行自底向上的分析，并在每条产生式归约时调用 ast.c 中的构造函数生成语法树结点。

本实验中使用的主要文法规则包括程序结构、变量声明、语句序列、赋值语句、复合语句、条件语句、循环语句以及表达式规则。程序整体结构如下：

program → PROGRAM ID ; vardec BEGIN stmts END .

变量声明部分支持多个变量一次声明，例如：

decl → idlist : type ;

idlist → ID | idlist , ID

type → INTEGER | REAL

通过该规则，程序可以识别如下声明：

```
VAR i, j, k: INTEGER;
f0: REAL;
```

语句部分主要包括赋值语句、if 语句、while 语句和 begin-end 复合语句，其核心规则如下：

```
stmt → assignstmt | ifstmt | whilestmt | compoundstmt
assignstmt → ID := exp
whilestmt → WHILE condition DO stmt
ifstmt → IF condition THEN stmt ELSE stmt
compoundstmt → BEGIN stmtseq END
```

表达式部分支持整数、实数、变量、括号表达式以及加减乘除运算：

```
exp → INT
exp → FLOAT
exp → ID
exp → exp + exp
exp → exp - exp
exp → exp * exp
exp → exp / exp
exp → ( exp )
```

为了避免表达式文法出现二义性，在 parser.y 中设置了运算符优先级和结合性。其中乘除优先级高于加减，加减和乘除均为左结合：

```
%left PLUS MINUS
%left TIMES DIVIDE
%right UMINUS
```

对于 if-then-else 结构中可能出现的悬挂 else 问题，使用如下优先级声明进行处理：

```
%nonassoc LOWER_THAN_ELSE
%nonassoc ELSE
```

这样可以保证 ELSE 默认与最近的未匹配 IF 结合，从而避免移进-归约冲突。

在抽象语法树构造方面，本实验使用 ast.h 中定义的数据结构表示程序、声明、

语句、表达式等语法成分。例如，程序结点包含程序名、声明列表和语句列表；声明结点包含类型和变量列表；语句结点根据不同类型分为赋值语句、if 语句、while 语句和复合语句；表达式结点则包括变量表达式、整数表达式、实数表达式和二元运算表达式。

在 parser.y 中，每个产生式都加入了相应动作。例如：

program 产生式归约时调用 A_Prog 构造整个程序的根结点；
变量声明产生式归约时调用 A_VarDec、A_DecList、A_VarList 等函数构造声明列表和变量列表；
赋值语句归约时调用 A_AssignStm 构造赋值语句结点；
while 语句归约时调用 A_WhileStm 构造循环语句结点；
if 语句归约时调用 A_IfStm 构造条件语句结点；
表达式归约时根据表达式类型分别调用 A_IntExp、A_RealExp、A_VarExp、A_OpExp 等函数。

最终，语法分析成功后，主程序调用 prast.c 中的打印函数 pr_prog，将构造出的语法树按照规定格式输出。

实验代码

parser.y：

第一种写法：

```
%{
#include <stdio.h>
#include <stdlib.h>
#include "util.h"
#include "errormsg.h"
#include "ast.h"
int yylex(void);
int parseSuccess = 0;
a_prog program;
int yydebug = 0;
void yyerror(char *s)
{
    parseSuccess = 0;
}
%}
%union {
```

```
    int ival;
    double fval;
    string sval;
    a_id id;
    a_exp exp;
    a_bexp bexp;
    a_stm stm;
    a_stm_list stmlist;
    a_var_list varlist;
    a_dec dec;
    a_dec_list declist;
    a_prog prog;
    ttype type;
}
%token <sval> ID
%token <ival> INT
%token <fval> FLOAT
%token INTEGER REAL
%token
    COMMA COLON SEMICOLON LPAREN RPAREN PERIOD
    PROGRAM BEGINN END VAR IF WHILE DO
    THEN ELSE
    ASSIGN EQ NEQ LT LE GT GE
    PLUS MINUS TIMES DIVIDE
%type <prog> program
%type <declist> vardec decls
%type <dec> decl
%type <varlist> idlist
%type <type> type
%type <stmlist> stmtseq
%type <stm> stmt
%type <exp> exp
%type <bexp> cond
%start program
%left PLUS MINUS
%left TIMES DIVIDE
%nonassoc LT LE GT GE EQ NEQ
%right THEN ELSE
%%
program
    : PROGRAM ID SEMICOLON vardec BEGINN stmtseq END PERIOD
      {  program =
        A_Prog(
            EM_tokPos,
```

```
            $2,
            $4,
            $6
        ); }
    ;
vardec
    : VAR decls
    {
    $$ = $2;
    }
    ;
decls
: decl           { $$ = A_DecList($1, NULL); }
| decl decls     { $$ = A_DecList($1, $2); }
;
decl
    : idlist COLON type SEMICOLON   { $$ = A_VarDec(EM_tokPos, $1, $3);
 }
    ;
type
    : INTEGER          { $$ = T_int; }
    | REAL             { $$ = T_real; }
    ;
idlist
: ID             { $$ = A_VarList(A_Id(EM_tokPos, $1), NULL); }
| ID COMMA idlist { $$ = A_VarList(A_Id(EM_tokPos, $1), $3); }
;
stmtseq
: stmt                 { $$ = A_StmList($1, NULL); }
| stmt SEMICOLON stmtseq { $$ = A_StmList($1, $3); }
;
stmt
:
ID ASSIGN exp
{
    $$ =
      A_Assign(
        EM_tokPos,
        A_Id(EM_tokPos,$1),
        $3
      );
}
|
IF cond THEN stmt ELSE stmt
```

```
{
    $$ =
      A_If(
        EM_tokPos,
        $2,
        $4,
        $6
      );
}
|
WHILE cond DO stmt
{
    $$ =
      A_While(
        EM_tokPos,
        $2,
        $4
      );
}
|
BEGINN stmtseq END
{
    $$ =
      A_Seq(
        EM_tokPos,
        $2
      );
}
;
cond
:
exp LT exp
{
    $$ = A_BExp(EM_tokPos,A_ltOp,$1,$3);
}
|
exp LE exp
{
    $$ = A_BExp(EM_tokPos,A_leOp,$1,$3);
}
|
exp GT exp
{
    $$ = A_BExp(EM_tokPos,A_gtOp,$1,$3);
```

```
}
|
exp GE exp
{
    $$ = A_BExp(EM_tokPos,A_geOp,$1,$3);
}
|
exp EQ exp
{
    $$ = A_BExp(EM_tokPos,A_eqOp,$1,$3);
}
|
exp NEQ exp
{
    $$ = A_BExp(EM_tokPos,A_neqOp,$1,$3);
}
;
exp
:
INT
{
    $$ = A_IntExp(EM_tokPos,$1);
}
|
FLOAT
{
    $$ = A_RealExp(EM_tokPos,$1);
}
|
ID
{
    $$ =
      A_VarExp(
        EM_tokPos,
        A_Id(EM_tokPos,$1)
      );
}
|
LPAREN exp RPAREN
{
    $$ = $2;
}
|
exp PLUS exp
```

```
{
    $$ = A_OpExp(EM_tokPos,A_plusOp,$1,$3);
}
|
exp MINUS exp
{
    $$ = A_OpExp(EM_tokPos,A_minusOp,$1,$3);
}
|
exp TIMES exp
{
    $$ = A_OpExp(EM_tokPos,A_timesOp,$1,$3);
}
|
exp DIVIDE exp
{
    $$ = A_OpExp(EM_tokPos,A_divideOp,$1,$3);
}
;
```

第二种写法：

```
%{
#include <stdio.h>
#include "ast.h" //该文件定义了抽象语法树（分析树）的数据结构
#include "errormsg.h"
#include "util.h"
#define YYDEBUG 1
/* 允许跟踪错误，与%debug 功能相同 */

int yylex(void); /* function prototype */

/* 该函数显示错误信息 s，显示时包含了错误发生的位置。*/
void yyerror(char *s)
{
 EM_error(EM_tokPos, "%s", s);
}
```

```
/* 存放抽象语法树中 "程序" 数据结构的变量，保存最终生成的语法树 */
a_prog program = NULL;

%}


 /* 定义属性值栈的类型，最后一个实验可能需要修改此处  */
%union {
       int ival;
        double fval;
        string sval;
       a_exp exp;
       a_stm stm;
       a_stm_list stms;
       a_dec dec;
       a_dec_list decl;
       a_var_list vlist;
       a_bexp bexp;
}

 /* 定义各个终结符，以及他们的属性值的类型，最后一个实验可能需要修改此处 */
%token <sval> ID /* id */
%token <ival> INT  /*整型数*/
%token <fval> FLOAT /*浮点数*/
%token INTEGER REAL  /*两种类型名：整型、实型*/
%token
  COMMA COLON SEMICOLON LPAREN RPAREN PERIOD /* 符号 , : ; ( ) . */
  PROGRAM BEGINN END VAR IF WHILE DO   /* 关键字：PROGRAM BEGIN END VAR
IF WHILE Do */
```

```
  /* 想想为什么不声明为 BEGIN?  */
  THEN ELSE /* 关键字：THEN ELSE */
  ASSIGN EQ NEQ LT LE GT GE /* 符号 :=    =  <>  <  <=  >  >= */
  PLUS MINUS TIMES DIVIDE /* 符号 + = * / */
%start program

  /* 定义各个非终结符的属性值类型，指定非终结符返回的类型  */
%type <decl> vardec declist
%type <dec> dec
%type <vlist> idlist
%type <stms> stmts stmtlist
%type <stm> stmt assignment_stmt if_stmt while_stmt compound_stmt
%type <exp> exp
%type <bexp> condition

/* 优先级处理 */
%nonassoc LOWER_THAN_ELSE
%nonassoc ELSE
%left EQ NEQ LT LE GT GE
%left PLUS MINUS
%left TIMES DIVIDE

%%
 /* 文法：program--> PROGRAM ID; vardec BEGIN stmts END.
  *       vardec-->   VAR declist
  *       declist-->  dec | declist dec
  *       dec-->  idlist:INTEGER; | idlist:REAL;
  *       idlist-->   ID | idlist,ID
  *       stmts--> stmtlist | (empty)
```

```
 *      stmtlist-->   stmt | stmtlist;stmt | stmtlist;
 *      stmt-->   assignment_stmt | if_stmt | while_stmt |
compound_stmt
 *      assignment_stmt-->   ID:=exp
 *      if_stmt-->   IF condition THEN stmt
 *                | IF condition THEN stmt ELSE stmt
 *      while_stmt-->   WHILE condition DO stmt
 *      compound_stmt-->   BEGINN stmts END
 *      condition-->   exp LT exp  | exp LE exp | exp NEQ exp |
 *                  | exp GT exp | exp GE exp | exp EQ exp
 *      exp--> exp PLUS exp | exp MINUS exp | exp TIMES exp
 *           | exp DIVIDE exp |  exp MINUS exp | LPAREN exp RPAREN
 *           | INT | FLOAT | ID
 * 最后一个实验需要修改这个文法
 */
program :       PROGRAM ID SEMICOLON vardec BEGINN stmts END PERIOD
                                        {program = A_Prog(EM_tokPos, $2,
$4, $6);}
                                ;

vardec  : VAR declist   {$$ = $2;}
       | /* empty */   { $$ = NULL; }
                                ;
declist : dec         { $$ = A_DecList($1, NULL); }
       | dec declist  { $$ = A_DecList($1, $2); }
       ;

dec     : idlist COLON INTEGER SEMICOLON { $$ = A_VarDec(EM_tokPos, $1,
T_int); }
       | idlist COLON REAL SEMICOLON    { $$ = A_VarDec(EM_tokPos, $1,
T_real); }
```

```
       ;

idlist  : ID            { $$ = A_VarList(A_Id(EM_tokPos, $1), NULL); }
       | ID COMMA idlist { $$ = A_VarList(A_Id(EM_tokPos, $1), $3); }
       ;

stmts   : /* empty */ { $$ = NULL; }
       | stmtlist    { $$ = $1; }
       ;

stmtlist: stmt                  { $$ = A_StmList($1, NULL); }
       | stmt SEMICOLON stmtlist { $$ = A_StmList($1, $3); }
       ;

stmt    : assignment_stmt { $$ = $1; }
       | if_stmt        { $$ = $1; }
       | while_stmt     { $$ = $1; }
       | compound_stmt   { $$ = $1; }
       ;

assignment_stmt : ID ASSIGN exp { $$ = A_Assign(EM_tokPos,
A_Id(EM_tokPos, $1), $3); }
              ;

if_stmt : IF condition THEN stmt %prec LOWER_THAN_ELSE    { $$ =
A_If(EM_tokPos, $2, $4, NULL); }
       | IF condition THEN stmt ELSE stmt              { $$ =
A_If(EM_tokPos, $2, $4, $6); }
       ;
```

```
while_stmt : WHILE condition DO stmt { $$ = A_While(EM_tokPos, $2,
$4); }
          ;

compound_stmt : BEGINN stmts END { $$ = A_Seq(EM_tokPos, $2); }
             ;

condition : exp EQ exp    { $$ = A_BExp(EM_tokPos, A_eqOp, $1, $3); }
          | exp NEQ exp   { $$ = A_BExp(EM_tokPos, A_neqOp, $1, $3); }
          | exp LT exp    { $$ = A_BExp(EM_tokPos, A_ltOp, $1, $3); }
          | exp LE exp    { $$ = A_BExp(EM_tokPos, A_leOp, $1, $3); }
          | exp GT exp    { $$ = A_BExp(EM_tokPos, A_gtOp, $1, $3); }
          | exp GE exp    { $$ = A_BExp(EM_tokPos, A_geOp, $1, $3); }
          ;
exp     : INT              { $$ = A_IntExp(EM_tokPos, $1); }
        | FLOAT            { $$ = A_RealExp(EM_tokPos, $1); }
        | ID               { $$ = A_VarExp(EM_tokPos, A_Id(EM_tokPos,
$1)); }
        | exp PLUS exp      { $$ = A_OpExp(EM_tokPos, A_plusOp, $1,
$3); }
        | exp MINUS exp     { $$ = A_OpExp(EM_tokPos, A_minusOp, $1,
$3); }
        | exp TIMES exp     { $$ = A_OpExp(EM_tokPos, A_timesOp, $1,
$3); }
        | exp DIVIDE exp    { $$ = A_OpExp(EM_tokPos, A_divideOp, $1,
$3); }
        | LPAREN exp RPAREN { $$ = $2; }
        ;
```

其中，第二种写法在 yyerror()函数中调用 EM_error(EM_tokPos, "%s", s)，相较第一种方式能够输出错误信息及错误位置，便于调试和定位语法错误，并且将赋值语句、条件语句、循环语句和复合语句分别定义为独立的非终结符，

使文法结构更加清晰，便于后续扩展。但它们最终都能够完成实验要求，完成语法分析并正确构造抽象语法树

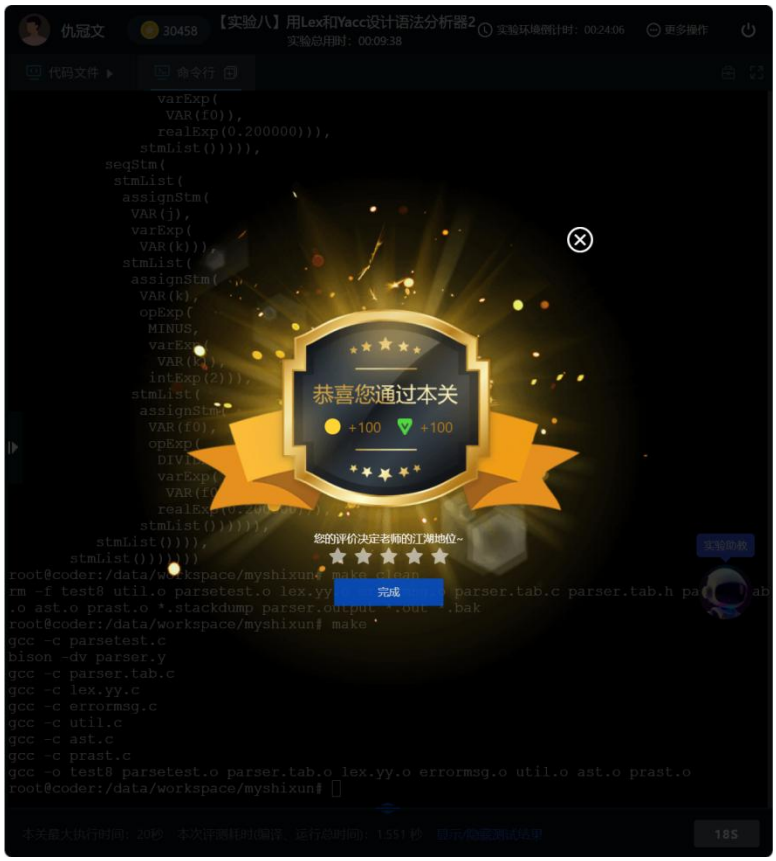
ast.c

```
/* make exp node from real */
a_exp A_RealExp(a_pos pos, double fval){
  a_exp ret = checked_malloc(sizeof(*ret));
  ret->kind = A_realExp;
  ret->pos = pos;
  ret->exp.fval = fval;
  return ret;}
/* make exp node from id node 变量表达式节点 */
a_exp A_VarExp(a_pos pos, a_id var){
  a_exp ret = checked_malloc(sizeof(*ret));
  ret->kind = A_varExp;
  ret->pos = pos;
  ret->exp.var = var;
  return ret;}
/* make binary operands exp node 二元算术运算节点 */
a_exp A_OpExp(a_pos pos, a_op op, a_exp left, a_exp right){
  a_exp ret = checked_malloc(sizeof(*ret));
  ret->kind = A_opExp;
  ret->pos = pos;
  ret->exp.biopExp.op = op;
  ret->exp.biopExp.left = left;
  ret->exp.biopExp.right = right;
  return ret;}
/* make boolean exp node */
a_bexp A_BExp(a_pos pos, a_bop bop, a_exp left, a_exp right){
  a_bexp ret = checked_malloc(sizeof(*ret));
```

```
  //布尔表达式无需设置类型
  ret->pos = pos;
  ret->bexp.bop = bop;
  ret->bexp.left = left;
  ret->bexp.right = right;
  return ret;}
/* make assign statement 赋值语句节点 */
a_stm A_Assign (a_pos pos, a_id var, a_exp exp){
  a_stm ret = checked_malloc(sizeof(*ret));
  ret->kind = A_assign;
  ret->pos = pos;
  ret->stm.assign.var = var;
  ret->stm.assign.exp = exp;
  return ret;}
/* make if statement IF 语句节点  */
a_stm A_If(a_pos pos, a_bexp b, a_stm s1, a_stm s2){
  a_stm ret = checked_malloc(sizeof(*ret));
  ret->kind = A_if;
  ret->pos = pos;
  ret->stm.iff.b = b;
  ret->stm.iff.s1 = s1;
  ret->stm.iff.s2 = s2;
  return ret;}
/* make while statement WHILE 语句节点*/
a_stm A_While(a_pos pos, a_bexp b, a_stm s){
  a_stm ret = checked_malloc(sizeof(*ret));
  ret->kind = A_while;
  ret->pos = pos;
  ret->stm.whilee.b = b;
```

```
  ret->stm.whilee.s = s;
  return ret;}
/* make sequence statement 顺序语句（复合语句）节点*/
a_stm A_Seq(a_pos pos, a_stm_list sl){
  a_stm ret = checked_malloc(sizeof(*ret));
  ret->kind = A_seq;
  ret->pos = pos;
  ret->stm.seq = sl;
  return ret; }
```

五、 实验结果



六、 实验中遇到的问题、难点及解决方案

问题 1：ast.c 中构造函数未完善

现象：提供的 ast.c 中多个函数体为空（有 checked_malloc 和 return ret），导致树节点字段全为 0/NULL。

解决方法：逐一补全所有构造函数，按照 ast.h 中的结构体字段赋值：

```
a_exp A_RealExp(a_pos pos, double fval) {
    a_exp ret = checked_malloc(sizeof(*ret));
    ret->kind = A_realExp;
    ret->pos = pos;
    ret->exp.fval = fval;
    return ret;}// ... 其余同理
```

问题 2：列表顺序颠倒

现象：decList 中变量声明顺序与源码相反，stmList 中语句顺序也颠倒 varList 中变量顺序倒置。

原因：初版文法使用左递归 + 新元素追加到头部：

```
declist : declist decitem   ← 新 decitem 追加在已有列表之后
        | decitem           ← 但构造时用 A_DecList($2, $1) 实际是反的
```

解决方法：改为右递归，保持源码顺序：

```
declist : decitem
        { $$ = A_DecList($1, NULL); }
        | decitem declist
        { $$ = A_DecList($1, $2); }
        ;


idlist  : id
        { $$ = A_VarList($1, NULL); }
        | id COMMA idlist
        { $$ = A_VarList($1, $3); }
        ;
```

```
stmts   : stmt
        { $$ = A_StmList($1, NULL); }
        | stmt SEMICOLON stmts
        { $$ = A_StmList($1, $3); }
        ;
```

问题 3：抽象语法树数据结构不一致

现象：编译时出现 unknown type name 'a_prog'、unknown type name 'a_stm_list'、结构体成员不存在等错误。

原因：实验八新增了 AST 构造和打印功能。平台默认的 parsetest.c 和 prast.c 使用的是 a_prog、a_dec_list、a_stm_list 等小写风格类型名，而部分 AST 文件中使用的是 A_prog、A_decList、A_stmList 等大写风格类型名，导致不同文件之间的数据结构无法对应。

解决方式：统一修改 ast.h、ast.c、parser.y 和 prast.c 中的 AST 类型名和结构成员名称，使语法分析器生成的语法树结点能够被打印函数正确访问。最终统一使用 a_prog、a_dec_list、a_var_list、a_stm_list、a_exp 等结构表示程序、声明、变量列表、语句列表和表达式结点。

问题 4：

现象：在命令行输入 make 命令后，出现如下报错：

```
root@coder:/data/workspace/myshixun# make
gcc -c parsetest.c
bison -dv parser.y
gcc -c parser.tab.c
gcc -c lex.yy.c
In file included from parser.l:8:0:
parser.y:28:9: error: unknown type name 'a_exp'
         a_exp exp;
         ^~~~
parser.y:29:9: error: unknown type name 'a_stm'
         a_stm stm;
         ^~~~
parser.y:30:9: error: unknown type name 'a_stm_list'
         a_stm_list stms;
         ^~~~~~~~~~
parser.y:31:9: error: unknown type name 'a_dec'
         a_dec dec;
         ^~~~
parser.y:32:9: error: unknown type name 'a_dec_list'
         a_dec_list decl;
         ^~~~~~~~~~
parser.y:33:9: error: unknown type name 'a_var_list'
         a_var_list vlist;
         ^~~~~~~~~~
parser.y:34:9: error: unknown type name 'a_bexp'
         a_bexp bexp;
         ^~~~~~
makefile:23: recipe for target 'lex.yy.o' failed
make: *** [lex.yy.o] Error 1
```

但检查 parser.y 文件后发现已经包含了 ast.h 文件。

原因：虽然在 parser.y 的顶部引用了 ast.h，但由于 Bison（Yacc）的工作机制：

- parser.tab.h 的生成：Bison 会根据%union 定义生成一个头文件 parser.tab.h。这个文件里包含了类似 a_exp exp; 这样的代码。

- Lex 的编译过程：当编译 lex.yy.c（也就是词法分析器）时，会引用 parser.tab.h 以获得 Token 的定义。

- 冲突点：如果在 lex.yy.c 中，包含 parser.tab.h 之前没有先包含 ast.h，编译器在处理 parser.tab.h 里的 a_exp 时，根本不知道 a_exp 是什么类型（因为 ast.h 还没被读到）。

解决方法：修改 parser.l 文件，在包含 parser.tab.h 之前，手动包含 ast.h

七、 感想和收获

本实验的核心是在文法产生式中添加语义动作构造 AST，分层设计程序、声明、语句、表达式各类语法树结点，借助 %union 实现标识符、数值与树节点的属性传递。

针对四则运算二义性、if 悬挂 else 冲突等问题，我学会通过优先级与结合性声明消除移进归约冲突；调试列表顺序颠倒 bu 时，分清左右递归对链表存储顺序的影响，掌握序列类语法结构的标准处理方式。也在实验过程中学习和了解 Lex 与 Yacc 头文件加载顺序的隐藏陷阱。

本次实验中的多文件连接问题和拆分独立非终结符的模块化文法写法，让我认识到代码分层和模块化对后续扩展的重要性。